Project 3 Field Guide: Thought Cascade

Companion Sheet — Read Alongside `p03_Thought_Cascade.ipynb` (No Separate Report Required)

ECE 4424 / CS 4824: Machine Learning
Instructor: Prof. Ming Jin · 100 pts (10%)

Big Picture & Golden Rule of Grading: In Projects 1–2, you built classical ML models and probed hidden LLM activations. In **Project 3**, you wrap a quantized 1.7B language model (`Qwen3-1.7B`) inside a bounded **ReAct code-repair loop** to test a core question in modern AI engineering: *When does test-runner feedback actually help an LLM fix bugs—and when does it fail?* **You are NEVER graded on how high the LLM scores.** You earn 100% credit through valid code contracts, clean experimental control, and honest analysis of your saved outputs—including unchanged patches, failed repairs, and `0/3` null results. Never re-run seeds to "fish" for a better outcome.

1. Recommended 3-Stage Workflow (Save Time & Protect Your Free GPU Quota)

Stage	Runtime Needed	What You Do	Ready to Move On When...
Stage 1: Build & Test Offline	**CPU only** (0 model calls, < 2 sec)	Run Setup. Complete the **6 `TODO` functions** (3 in Q1, 1 in Q4, 1 in Q5, `react_loop` in Q6). Run the deterministic `ScriptedClient` checks before loading any GPU model.	`check_q1` , `check_components` , and `check_loop` all print `passed` .
Stage 2: Run GPU Experiments	**Colab T4 GPU** (or CUDA / Apple MPS; ~10–15 min)	Select **T4 GPU**, run Setup, then execute Q1 → Q6 once in order using `SEEDS = (11, 22, 33)` (at most **57 model calls** total: 6 in Q1, 3 in Q3, up to 48 in Q6).	All tables, plots, paired checks, and `results/student_run.json` are saved.
Stage 3: Write & Export Single PDF	**No GPU needed** (0 model calls)	Write **Q2** (200–350w), **Q3** (100–200w), and **Q6** (250–400w) directly in the Markdown cells. Save/download `.ipynb` , run `export_submission()` , open HTML in Chrome → **Print > Save as PDF**.	`LastName_FirstName_Project3.pdf` shows all 6 functions, 3 write-ups, tables & figures unclipped.

2. Mini Knowledge Points — Part I: Tests, Metrics, & Failure Modes (Q1–Q3)

KP 1 · Independent Test Oracles & Half-Open Intervals `[left, right)` (Q1)

- **Why use an independent oracle?** Never verify a fast, tricky algorithm using another copy of the same tricky logic. Pair the target function with a dead-simple **oracle** (e.g., an `O(n)` left-to-right scan) whose correctness is obvious, then use that oracle to label both hand-written boundary inputs and seeded random ("fuzz") inputs.
- **Half-open interval convention `[left, right)` :** Many search routines represent the active window as `[left, right)` , where `left` is *included* and `right` is *excluded* (so `0, len(arr)` covers indices `0 .. len(arr)-1` , and `left == right` means the window is empty). Whenever a branch updates `left` or `right` , ask: *Does this update preserve the half-open convention without skipping a candidate index, and does matching `target` stop too early when duplicates exist?*

KP 2 · Two Different "Rates" & Sharper Ruler vs. Changed Model Behavior (Q1–Q2)

- **Case Pass Rate (`pass_rate`) ≠ Task Success Rate (`all_passed` / `success_rate`):** If *one* generated patch passes 90 of 100 test cases, that is **1 model attempt evaluated on 100 inputs**—its case pass rate is `0.90` , while its task success is `False` (`0.0`). Do not confuse 100 test inputs with 100 LLM attempts!
- **Measurement change vs. model change:** Re-scoring the *exact same saved code* on `STRICT` after `MINIMAL` changes your **measurement instrument**, not what the model generated. Always inspect `code_changed` too: a patch can copy the starter verbatim (`False`), or modify text (`True`) without fixing the bug.

KP 3 · Three Failure Layers — Why `status == 'tested'` Does NOT Mean "No Exceptions" (Q2)

Failure Layer	What Actually Happened	How to Spot It in Saved Observations
1. Syntax / Compile Error	Invalid Python grammar; module could not be parsed/compiled, so zero tests ran.	`compilation == 'error'`, `status == 'syntax_error'`, plus syntax details in `error`.
2. Runtime Exception	Valid syntax; runner finished the suite (`status == 'tested'`), **but** individual test calls crashed (e.g., `RecursionError`, `IndexError`). *(Timeouts/blocked imports give `status == 'timeout'` or `'rejected'`.)*	`compilation == 'success'`, `runtime_errors > 0`, and per-case `Error:` lines inside `failures`.
3. Logical Wrong Answer	Function returned normally without raising an exception, but returned the wrong value (or mutated its input list).	`wrong_answers > 0` (or `input_mutations > 0`); compare `Expected` vs. `Actual` in `failures`.

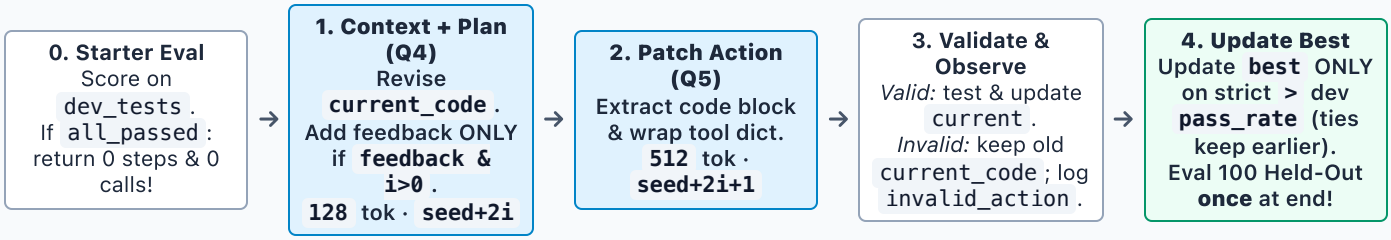
Project 3 Field Guide: Controller, Experiments & Checklist

Q4–Q6 Mental Models, Common Contract Pitfalls, and Final PDF Export Check

2. Mini Knowledge Points — Part II: ReAct Loop & Controlled Experiment (Q4–Q6)

KP 4 · Two Pointers in `react_loop` (`Current` vs. `Best`) & Dev vs. Held-Out Hygiene (Q4–Q6)

Multi-round LLM edits are **not monotone**: Round 2 can easily break a partial fix from Round 1! Your controller must maintain **two separate candidates** while keeping held-out tests completely out of the loop:



Analogy to ML Training: `dev_tests` = Validation set (drives Round 2 feedback, early stopping, and checkpoint selection). `100 held-out cases` = Locked Test set (evaluated once after selection; never placed in prompts or used to tune).

KP 5 · Paired Seeds, Information vs. Reasoning Bottlenecks, & Cost Accounting (Q3 & Q6)

- **Why Round 1 MUST match across conditions:** In Round 1 (`i = 0`), neither `feedback=False` nor `feedback=True` sees test results (`observation=None`), and both use identical seeds. Thus, for each seed, Round 1's prompt, plan, and patch must be **identical** (`same_first_* == True`)—guaranteeing that any Round 2 divergence is caused *solely* by test feedback!
- **Compare what each Q6 task lacks:**
  - *Binary Search:* The specification already states the full requirement; failing tests only show concrete counterexamples of a rule the model already saw (testing whether feedback overcomes a **reasoning/editing bottleneck**).
  - *Device-Status Decoding:* The prompt lists valid codes and labels *without* their correspondence (and listing order does not define it); failing development tests supply **missing external ground truth** absent from the initial prompt.
- **Uncertainty & Real Compute Cost:** With `3` seeds, distinguish **floor effects** (both 0%) from **ceiling effects** (both 100%), avoid over-claiming from 2 tasks or 1 fixed device mapping, and compare **input/output tokens and runtime seconds**—same max rounds does not mean equal token cost when feedback lengthens prompts!

3. Concrete Contract Guardrails & Common Traps to Avoid

Exercise	What We Expect (Do This)	Common Trap (Avoid This)
Q1: `build_strict_tests`	Put `>= 10` distinct explicit cases **first**, then append **exactly** `n_random` cases via local `random.Random(seed)`. Include empty list, singleton (**both** target present & absent), **unique** first & last index, absent below/between/above, negatives & duplicates.	Calling global `random.seed()`, deduplicating across the combined list (which breaks `len(explicit) + n_random`), or writing only one singleton case.
Q1: `measure_...`	Treat `records` as read-only; keep failed/non-compiling attempts in the denominator (`len(records)`); `raise ValueError` if `records` is empty.	Mutating dicts inside `records` or dropping failed attempts from averages.
Q4 & Q5: Plan & Action	Include `context` unchanged (plus `thought` in Q5). Keep prompts **100% task-agnostic** (`max_new_tokens=128` in Q4, `512` in Q5). Build `{'tool': 'fix_code', 'params': {'code': ...}}` in Python.	Hard-coding function names, bug hints, or status-code mappings in prompts (staff inspect Q4/Q5 code as well as scripted checks!).
Q6: `react_loop`	Call `validate_action(action)` **before** reading `action['params']['code']`. Update `best_code` only when `current_obs['pass_rate'] > best_obs['pass_rate']`.	Overwriting `current_code` on an invalid action, leaking feedback in Round 1 (`i=0`) or `feedback=False`, or returning the final patch instead of best dev patch.

4. Pre-Submission Checklist (Submit One File: `LastName_FirstName_Project3.pdf`)

- ☐ **Six `TODO` functions completed in notebook cells:** `check_q1`, `check_components`, and `check_loop` all pass without modifying files inside `.thought_cascade/` (Setup overwrites extracted helpers!).
- ☐ **Complete controlled GPU run (≤ 57 model calls):** All tables, trajectory plots, and code diffs are visible; Q6 paired first-round checks (`same_first_prompt`, `same_first_plan`, `same_first_patch`) are all `True`. Keep `.ipynb` and `results/student_run.json` as personal backups.
- ☐ **Three written analyses in Markdown cells: Q2** (200–350 words), **Q3** (100–200 words), and **Q6** (250–400 words) address every rubric bullet using your actual tables and saved logs (no selective re-runs).
- ☐ **Clean PDF export:** Saved/downloaded completed `.ipynb` → ran `export_submission()` → opened `*_submission.html` in Chrome → **Print > Save as PDF**, and checked every page to confirm all 6 functions, 3 write-ups, tables, and figures appear cleanly without clipping.

If no free GPU is available after passing Stage 1 checks, contact course staff for recorded run logs and label them clearly.